

RigidBody Collisions in 2D

For the exercises that follow, implement the code into a file called `rigidbody2d_collision.py` and hand that in on avenue. For the written answers and discussion, hand-in a separate pdf file with your answers.

Exercise 1: Complete your `rigidbody2d_collision.py` code as discussed and worked on in class. (see `VG3_CollisionDetection.pdf` lecture notes). You should have begun with `rigidbody2d_collison_hack.py` from Avenue or similar. In that file, the `Particle` and `RigidBody2D` classes should be set up as follows:

```
Particle(NodePath):  
  
    def __init__(self, *args, **kwargs):  
        NodePath.__init__(self, *args, **kwargs)  
        self.vel = Vec3(0.,0.,0.)  
        self.inverseMass = 1.0  
        self.radius = 1.0  
        self.collisionRadius = 1.8  
  
class RigidBody2D(Particle):  
    def __init__(self, *args, **kwargs):  
        Particle.__init__(self, *args, **kwargs)  
        self.theta = 0.0  
        self.omega = 0.0  
        self.inverseMomentOfInertia = self.inverseMass/(2./3.*self.radius**2)
```

The provided code was set up to generate an off-centre collision (see below). You will need to add code to handle more general collisions as discussed in the notes and worked on in class last week. For all the exercises here, use a single collision point that is the average of the contacts in the case of more than one contact. The most important part is to average to the contact location. You should only ever get this case with a face-face collision (2 contacts).

Exercises 2 (a): Collision cases with coefficient of Restitution $e=1$.

```
self.RigidBody2Ds = []  
for i in range(2):  
    p = RigidBody2D("p")  
    self.cube.instanceTo(p)  
  
    #Adjust physical and rendering properties of particle here  
    p.setPos(Vec3(-6+17.*i,25,i-0.5))  
    p.vel = Vec3(2-i*6.,0., 0.)
```

Above is the set up for the default collision. It should produce effectively similar outcomes to the hack version, but it should detect the collision instead of making assumptions the way the hack code did.

The case above is scenario 0 (default). Implement the following additional four cases for a total of five collisions. Your code should be able to run different scenarios by changing a variable called `scenario` from 0 to 4. Note that we are working in 2 dimensions here. Thus rotations always refer to rotations in the x-z plane

(around the y-axis). You can use the variables theta and omega to accomplish this. Note that both theta and omega use radians, not degrees.

Scenario 0: Default above: no initial rotation

Scenario 1: Rotate initial angle of body 0 to 30 degrees.

Scenario 2: Rotate initial angle of body 0 to 60 degrees and body 1 to 30 degrees.

Scenario 3: Set initial angular velocity of body 0 to -5 radians/sec. Set the initial velocity of body 1 to zero and put it at $x=-1.9$, $y=25$, $z=1.9$.

Scenario 4: Set initial angular velocity of body 0 to +3 radians/sec. Set the initial angle of body 1 to -10 degrees.

For each of the five cases record the time, point of collision (3d vector location) and normal at the point of collision for all collisions. Indicate if it was originally a single contact or if the collision involves simultaneous contacts that were averaged together. Indicate which body provided the corner and which body provides the face (edge). In the case of collision with multiple contacts, describe all of the contacts (before and after they are averaged). If the collision is complicated in some sense, describe it in detail.

Exercises 2 (b): Collision cases with coefficient of Restitution $e=0$.

Repeat the above collisions with $e=0$ and report the details as for 2 (a). You should describe the collisions in detail with words. You only need to indicate the time and properties of the first contact.

Exercise 3 Calculate the momentum, angular momentum and energy involved in the five collisions for each of the two cases (a) $e=1$ (b) $e=0$

Compute the initial and final momentum, energy and angular momentum before and after each collision. You will need to consider angular momentum and energy in the form of both bulk motion and rotation about the centre of mass of each body. Report the numerical results your hand-in (pdf file) to as many figures as printed by python print. You can look at the VG3_Rigidbody slides for the expressions for energy etc...

Indicate the value of the angular momentum and energy before and after each collision. Explicitly discuss how the energy has been transformed (from what to what) in each case. Discuss when a total quantity is conserved or not before and after a collision.